

Common Mistakes

常见错误

本页面主要列举一些竞赛中很多人经常会出现的错误。

因环境不同导致的错误

- `scanf` 或 `printf` 使用 `%I64d` 格式指示符在 Linux 下可能导致输出格式错误。

会引起 CE 的错误

这类错误多为词法、语法和语义错误，引发的原因较为简单，修复难度较低。

例：

- `int main()` 写为 `int mian()` 之类的拼写错误。
- 写完 `struct` 或 `class` 忘记写分号。
- 数组开太大，（在 OJ 上）使用了不合法的函数（例如多线程），或者函数声明但未定义，会引起链接错误。
- 函数参数类型不匹配。
 - 示例：如使用 `<LTAgorithm>` 头文件中的 `max` 函数时，传入了一个 `int` 类型参数和一个 `long long` 类型参数。

```
1 // query 为返回 long long 类型的自定义函数
2 printf("%lld\n", max(0, query(1, 1, n, 1, r)));
3
4 // 错误    没有与参数列表匹配的 重载函数 "std::max" 实例
```

- 使用 `goto` 和 `switch-case` 的时候跳过了一些局部变量的初始化。

不会引起 CE 但会引起 Warning 的错误

犯这类错误时写下的程序虽然能通过编译，但大概率会得到错误的程序运行结果。这类错误会在使用 `-W{warningtype}` 参数编译时被编译器指出。

- 赋值运算符 `=` 和比较运算符 `==` 不分。

- 示例：

```
1 std::srand(std::time(nullptr));
2 int n = std::rand();
3 if (n = 1)
4     printf("Yes");
5 else
6     printf("No");
7
8 // 无论 n 的随机所得值为多少，输出肯定是 Yes
9 // 警告    运算符不正确：在 Boolean 上下文中执行了常量赋值。应考虑改用「==」。
```

- 如果确实想在原应使用 `==` 的语句里使用 `=`（比如 `while (foo = bar)`），又不想收到 Warning，可以使用

双括号：while ((foo = bar))。

- 由于运算符优先级产生的错误。

- 示例：

```
1 // 错误
2 // std::cout << (1 << 1 + 1);
3 // 正确
4 std::cout << ((1 << 1) + 1);
5
6 // 警告      「<<」：检查运算符优先级是否有可能的错误；使用括号阐明优先级
```

- 不正确地使用 static 修饰符。
- 使用 scanf 读入的时候没加取地址符 &。
- 使用 scanf 或 printf 的时候参数类型与格式指定符不符。
- 同时使用位运算和逻辑运算符 == 并且未加括号。

- 示例：(x >> j) & 3 == 2

- int 字面量溢出。

- 示例：long long x = 0x7f7f7f7f7f7f7f7f, 1<<62。

- 未初始化局部变量。

▼ 未初始化变量会发生什么

原文：<https://loj.ac/d/3679> by @hly1204

例如我们在 C++ 中声明一个 int a; 但不初始化，可能有时候会认为 a 是一个「随机」（其实可能不是真的随机）的值，但是可能将其认为是一个固定的值，但实际上并非如此。

我们在简单的测试代码中

<https://wandbox.org/permlink/T2uiVe4n9Hg4EyWT>

代码是：

```
1 #include <Istream>
2
3 int main() {
4     int a;
5     std::cout << std::boolalpha << (a < 0 || a == 0 || a > 0);
6     return 0;
7 }
```

在一些编译器和环境上开启优化后，其输出为 false。

有兴趣的话可以看 <https://www.ralfj.de/blog/2019/07/14/uninit.html>，尽管其是用 Rust 做的实验，但是本质是一样的。

- 局部变量与全局变量重名，导致全局变量被意外覆盖。（开 -Wshadow 就可检查此类错误。）
- 运算符重载后引发的输出错误。
- 示例：

```

1 // 本意：前一个 << 为重载后的运算符，表示输出；后一个 << 为移位运算符，表示将 1
2 // 左移 1 位。 但由于忘记加括号，导致编译器将后一个 <<
3 // 也判作输出运算符，而导致输出的结果与预期不同。 错误 std::cout << 1 << 1; 正确
4 std::cout << (1 << 1);

```

既不会引起 CE 也不会引发 Warning 的错误

这类错误无法被编译器发现，仅能自行查明。

会导致 WA 的错误

- 上一组数据处理完毕，读入下一组数据前，未清空数组。
- 读入优化未判断负数。
- 所用数据类型位宽不足，导致溢出。
 - 如习语「三年 OI 一场空，不开 long long 见祖宗」所描述的场景。选手因为没有在正确的地方开 long long（将整数定义为 long long 类型），导致得出错误的答案而失分。
- 存图时，节点编号 0 开始，而题目给的边中两个端点的编号从 1 开始，读入的时候忘记 -1。
- 大/小于号打错或打反。
- 在执行 `ios::sync_with_stdio(false);` 后混用 `scanf/printf` 和 `std::cin/std::cout` 两种 IO，导致输入/输出错乱。
 - 示例：

```

1 // 这个例子将说明关闭与 stdio 的同步后，混用两种 IO 方式的后果
2 // 建议单步运行来观察效果
3 #include <LTcstdio>
4 #include <LTiostream>
5
6 int main() {
7     // 关闭同步后，cin/cout 将使用独立缓冲区，而不是将输出同步至 scanf/printf
8     // 的缓冲区，从而减少 IO 耗时
9     std::ios::sync_with_stdio(false);
10    // cout 下，使用 '\n' 换行时，内容会被缓冲而不会被立刻输出
11    std::cout << "a\n";
12    // printf 的 '\n' 会刷新 printf 的缓冲区，导致输出错位
13    printf("b\n");
14    std::cout << "c\n";
15    // 程序结束时，cout 的缓冲区才会被输出
16    return 0;
17 }

```

- 由于宏的展开，且未加括号导致的错误。
 - 示例：该宏返回的值并非 而是 。

```

1 #define square(x) x* x
2 printf("%d", square(2 + 2));

```

- 哈希的时候没有使用 unsigned 导致的运算错误。
 - 对负数的右移运算会在最高位补 1。参见：[位运算](#)

- 没有删除或注释掉调试输出语句。

- 误加了 ;。

- 示例：

```
1 /* clang-format off */
2 while (1);
3     printf("OI Wiki!\n");
```

- 哨兵值设置错误。例如，平衡树的 0 节点。

- 在类或结构体的构造函数中使用：初始化变量时，变量声明顺序不符合初始化时候的依赖关系。

- 成员变量的初始化顺序与它们在类中声明的顺序有关，而与初始化列表中的顺序无关。参见：[构造函数与成员初始化器列表](#) 的「初始化顺序」

- 示例：

```
1 #include <LTIostream>
2
3 class Foo {
4 public:
5     int a, b;
6
7     // a 将在 b 前初始化，其值不确定
8     Foo(int x) : b(x), a(b + 1) {}
9 };
10
11 int main() {
12     Foo bar(1, 2);
13     std::cout << bar.a << ' ' << bar.b;
14 }
15
16 // 可能的输出结果：-858993459 1
```

- 并查集合并集合时没有把两个元素的祖先合并。

- 示例：

```
1 f[a] = b;                // 错误
2 f[find(a)] = find(b);    // 正确
```

- freopen 使用 a 进行追加写

- CCF 的检测环境不会清空输出文件，使用 a 会导致上一位选手的输出也被评测机读入引发 WA

换行符不同

▼ Warning

在正式比赛中会尽量保证选手答题的环境和最终测试的环境相同。

本节内容仅适用于模拟赛等情况，而我们也建议出题人尽量让数据符合 [数据格式](#)。

不同的操作系统使用不同的符号来标记换行，以下为几种常用系统的换行符：

- LF（用 \n 表示）：Unix 或 Unix 兼容系统

- CR+LF (用 `\r\n` 表示) : Windows
- CR (用 `\r` 表示) : Mac OS 至版本 9

而 C/C++ 利用转义序列 `\n` 来换行, 这可能会导致我们认为输入中的换行符也一定是由 `\n` 来表示, 而只读入了一个字符来代表换行符, 这就会导致我们没有完全读入输入文件。

以下为解决方案:

- 多次 `getchar()`, 直到读到想要的字符为止。
- 使用 `cin` 读入, 这可能会增大代码常数。
- 使用 `scanf("%s", str)` 读入一个字符串, 然后取 `str[0]` 作为读入的字符。
- 使用 `scanf(" %c", &c)` 过滤掉所有空白字符。

会导致未知的结果

未定义行为会导致未知的结果, 可能是 WA, RE 等。编译器通常会假定你的程序不会出现未定义行为, 因此出现开 O2 与不开 O2 代码行为不一致的情况。

- 除以 0 (求 0 的逆元)

▼ 示例

```
1 cout << x / 0 << endl;
```

- 数组 (下标) 越界

例如:

- 未正确设置循环的初值导致访问了下标为 -1 的值。
- 无向图边表未开 2 倍。
- 线段树未开 4 倍空间。
- 看错数据范围, 少打一个零。
- 错误预估了算法的空间复杂度。
- 写线段树的时候, `pushup` 或 `pushdown` 叶节点。

正确的做法: 不要越界, 记得检查自己的代码, 使得下标访问数 x , 在定义的下标中。

- 除 main 外有返回值函数执行至结尾未执行任何 `return` 语句

即使有一个分支有返回值, 但是其他分支却没有, 结果也是未定义的。

可以向编译选项中追加 `-Wall`, 检查编译器是否给出有关于函数未 `return` 的警告。

- 尝试修改字符串字面量

▼ 示例

```
1 char *p = "OI-wiki";
2 p[0] = 'o';
3 p[1] = 'i';
```

这样试图修改字符串字面量会导致 **未定义行为**, 应当使用其他 **合适** 的数据类型, 例如 `std::string` 和 `char[]`。

- 多次释放/非法解引用一片内存

例如：

- 未初始化就解引用指针。
- 指针指向的内存区域已经释放。

使用 `erase` 或 `delete` 或 `free` 操作应注意不要对同一地址/对象多次使用。

- 尝试释放由 `new []` 分配的整块内存的一部分

例如：

```
1 object *pool = new object[POOL_SIZE];
2
3 object *pointer = pool + 10;
4
5 // 报错!
6 delete pointer;
```

常见于使用内存池提前分配整块内存后，试图使用 `delete` 或 `free()` 释放从内存池中获取的单个对象。

- 解引用空指针/野指针

对于空指针：先应该判断空指针，可以用 `p == nullptr` 或 `!p`。

对于野指针：可以释放指针的时候将其置为 `nullptr` 以规避。

- 有符号数溢出

例如我们有一个表达式 `x+1 > x`。

正常输出应当是 `true`，但是在 `INT_MAX` 作为 `x` 时输出 `false`，这时称为 `signed integer overflow`。

可以使用更大的数据类型（例如 `long long` 或 `__int128`），或判断溢出。若保证无负数，亦可使用无符号整型。

有符号整数溢出可能影响编译优化，例如代码：

```
1 int foo(int x) {
2     if (x > x + 1) return 1;
3     return 0;
4 }
```

可能被编译器直接优化为：

```
1 int foo(int x) { return 0; }
```

因为编译器可以假定有符号整数永远不会溢出，因此 `x > x + 1` 永远不会成立。

- 使用未初始化的变量

▼ 示例

```
1 int foo(int a) {
2     int t; /* 没有初始化 */
3     if (/* 使用 */ t > 3) return a;
4     return 0;
5 }
```

会导致 RE

- 没删文件操作（某些 OJ）。
- 排序时比较函数的错误 `std::sort` 要求比较函数是严格弱序：`a<a` 为 `false`；若 `a<b` 为 `true`，则 `b<a` 为 `false`；若 `a<b` 为 `true` 且 `b<c` 为 `true`，则 `a<c` 为 `true`。其中要特别注意第二点。如果不满足上述要求，排序时很可能会 RE。例如，编写莫队的奇偶性排序时，这样写是错误的：

```
1 bool operator<(const int a, const int b) {
2     if (block[a.l] == block[b.l])
3         return (block[a.l] & 1) ^ (a.r < b.r);
4     else
5         return block[a.l] < block[b.l];
6 }
```

上述代码中 `(block[a.l]&1)^(a.r<b.r)` 不满足上述要求的第二点。改成这样就正确了：

```
1 bool operator<(const int a, const int b) {
2     if (block[a.l] == block[b.l])
3         // 错误：不满足严格弱序的要求
4         // return (block[a.l] & 1) ^ (a.r < b.r);
5         // 正确
6         return (block[a.l] & 1) ? (a.r < b.r) : (a.r > b.r);
7     else
8         return block[a.l] < block[b.l];
9 }
```

- Windows 下栈空间不足，导致栈空间溢出，Windows 向程序发出 SIGSEGV 信号，程序终止并返回 3221225725（即 0xC00000FD，NTSTATUS 定义为 `STATUS_STACK_OVERFLOW`）。若使用 gcc 编译器，可在编译时加入命令 `-Wl,--stack=SIZE` 以指定栈空间大小限制，其中 `SIZE` 为栈空间大小字节数。

Linux 下栈空间不足，导致栈空间溢出，Linux 会在栈堆中乱写 `head_info`，此操作绝大多数情况下会导致程序立即退出，显示段错误（核心已转储）/segmentation fault (core dumped) 等字样。

可以在终端下使用 `ulimit -s SIZE` 修改当前终端的栈空间限制，其中 `SIZE` 为栈空间大小千字节数（KB）。
请注意，如果你将栈空间限制设置过大，无穷递归可能导致递归栈过大进而导致系统崩溃。

会导致 TLE

- 分治未判边界导致死递归。
- 死循环。
 - 循环变量重名。
 - 循环方向反了。
- BFS 时不标记某个状态是否已访问过。
- 使用宏展开编写 `min/max`

这种错误会大大增加程序的运行时间，甚至直接影响代码的时间复杂度。在初学者写线段树时尤为多见。

常见的错误写法是这样的：

```
1 #define Min(x, y) ((x) < (y) ? (x) : (y))
2 #define Max(x, y) ((x) > (y) ? (x) : (y))
```

这样写虽然在正确性上没有问题，但是如果直接对函数的返回值取 `max`，如 `a = Max(func1(), func2())`，而这个函数的运行时间较长，则会大大影响程序的性能，因为宏展开后是 `a = func1() > func2() ? func1() : func2()` 的形式，调用了三次函数，比正常的 `max` 函数多调用了一次。注意，如果 `func1()` 每次返回的答案不一样，还会导致这种 `max` 的写法出现错误。例如 `func1()` 为 `return ++a;` 而 `a` 为全局变量的情况。

示例：如下代码会被卡到单次查询 导致 TLE。

```
1 #define max(x, y) ((x) > (y) ? (x) : (y))
2
3 int query(int t, int l, int r, int ql, int qr) {
4     if (ql <= l && qr >= r) {
5         ++ti[t]; // 记录结点访问次数方便调试
6         return vi[t];
7     }
8
9     int mid = (l + r) >> 1;
10    if (mid >= qr) return query(lt(t), l, mid, ql, qr);
11    if (mid < ql) return query(rt(t), mid + 1, r, ql, qr);
12    return max(query(lt(t), l, mid, ql, qr), query(rt(t), mid + 1, r, ql, qr));
13 }
```

- 使用 `+` 运算符向 `std::string` 类字符串追加字符

这种错误会创建一个临时 `string` 变量，修改完成后再赋值给原变量。这种错误无法被编译器优化，在数据量大的情况下可能会导致时间复杂度退化。

常见 错误写法：

```
1 std::string a;
2 char b = 'c';
3 a = a + b;
```

当执行这段代码时，程序首先会创建一个临时 `string` 变量，随后将 `a` 的值存入临时变量，然后在末尾添加 `b` 的值，最后再存入 `a`。

从 [汇编结果](#) 可以看出，`a = a + b` 调用了三次 `std::__cxx11::basic_string` 中的功能，分别为 `operator+`、`operator=` 和创建变量。

正确写法应该是：

```
1 std::string a;
2 char b = 'c';
3 a += b;
```

[这种写法](#) 会直接将字符 `b` 附加到字符串 `a` 中，仅调用了一次 `operator+=`。更详细的性能比较可参考 [Benchmark](#)。

- 没删文件操作（某些 OJ）。
- 在 `for/while` 循环中重复执行复杂度非 的函数。严格来说，这可能会引起时间复杂度的改变。

会导致 MLE

- 数组过大。
 - ▶ Linux 下的内存占用指标详解
 - STL 容器中插入了过多的元素。

- 经常是在一个会向 STL 插入元素的循环中死循环了。
- 也有可能被卡了。

会导致常数过大

- 定义模数的时候，未定义为常量。
 - 示例：

```
1 // int mod = 998244353;      // 错误
2 const int mod = 998244353;  // 正确，方便编译器按常量处理
```

- 使用了不必要的递归（尾递归不在此列）。
- 将递归转化成迭代的时候，引入了大量额外运算。

只在程序在本地运行的时候造成影响的错误

- 文件操作有可能会发生的错误：
 - 对拍时未关闭文件指针 `fclose(fp)` 就又令 `fp = fopen()`。这会使得进程出现大量的文件野指针。
 - `freopen()` 中的文件名未加 `.in/.out`。
- 使用堆空间后忘记 `delete` 或 `free`。

参考资料与注释

1. [What is RSS and VSZ in Linux memory management - Stack Overflow↩](#)
 2. [Need explanation on Resident Set Size/Virtual Size - Stack Overflow↩](#)
 3. [虚拟内存 ↩](#)
 4. [常驻集大小 ↩](#)
-

本页面最近更新：2025/4/1 20:27:50, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[H-J-Granger](#), [StudyingFather](#), [broken-paint](#), [CarvingAn](#), [Chrogeek](#), [ChungZH](#), [Enter-tainer](#), [Estrella-Explore](#), [lr1d](#), [ksyx](#), [Marcythm](#), [NachtgeistW](#), [orzAtalod](#), [shuzhouliu](#), [yiyangit](#), [66Leo66](#), [Allenyou1126](#), [alphagocc](#), [bear-good](#), [caijianhong](#), [CCXXXI](#), [CodeZhangBorui](#), [CodingOler](#), [countercurrent-time](#), [EarlyOv0](#), [FinParker](#), [greyqz](#), [HeRaNO](#), [hsfzLZH1](#), [HuangYiming0608](#), [i-yyi](#), [imba-tjd](#), [inclyc](#), [isdanni](#), [MapMaths](#), [mgt](#), [ouuan](#), [SukkaW](#), [Sweetlemon68](#), [swiftqwq](#), [Tiphereth-A](#), [Xeonacid](#), [ylxmf2005](#), [zryi2003](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用